

操作系统

第二章 进程管理

Unix V6++ 的内存管理子系统

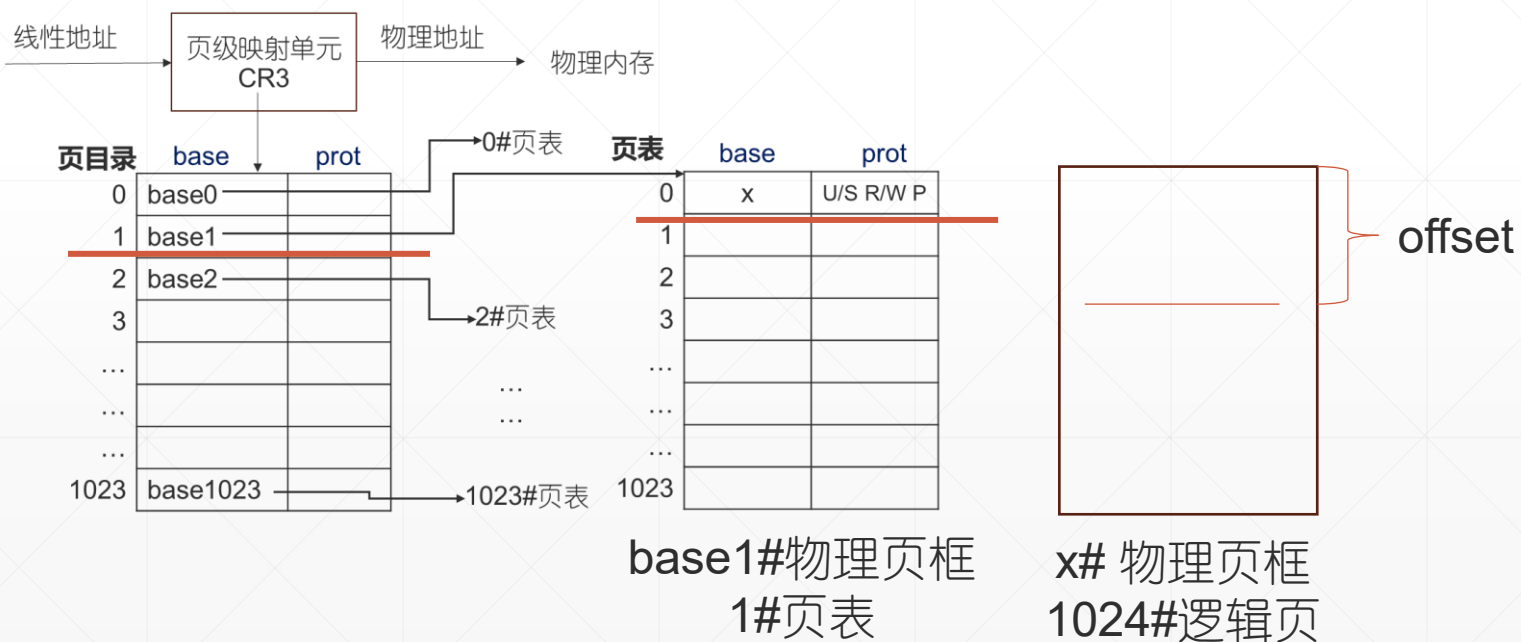
同济大学计算机系

页级地址映射和内存保护 (1)



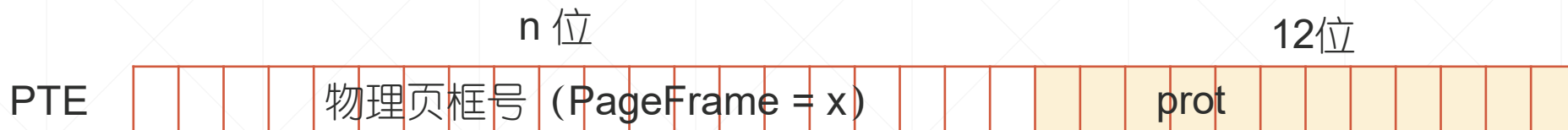
1、查页表

- 用页目录号 **pagetable** 查页目录
 - null**, 非法内存访问
 - 非空, 映射需要的页表在 **base1#物理页框**
- 用页号 **page** 查页表 (**base1#物理页框**)
 - null**, 非法内存访问
 - 非空, 获得映射所需的页表项





页级地址映射和内存保护 (2)



2、内存保护（检测prot中的bit）

CS



CPU用户态运行

if(CPU用户态运行 & U/S为0)

非法内存访问。应用程序无权访问内核，立即终止现运行进程。

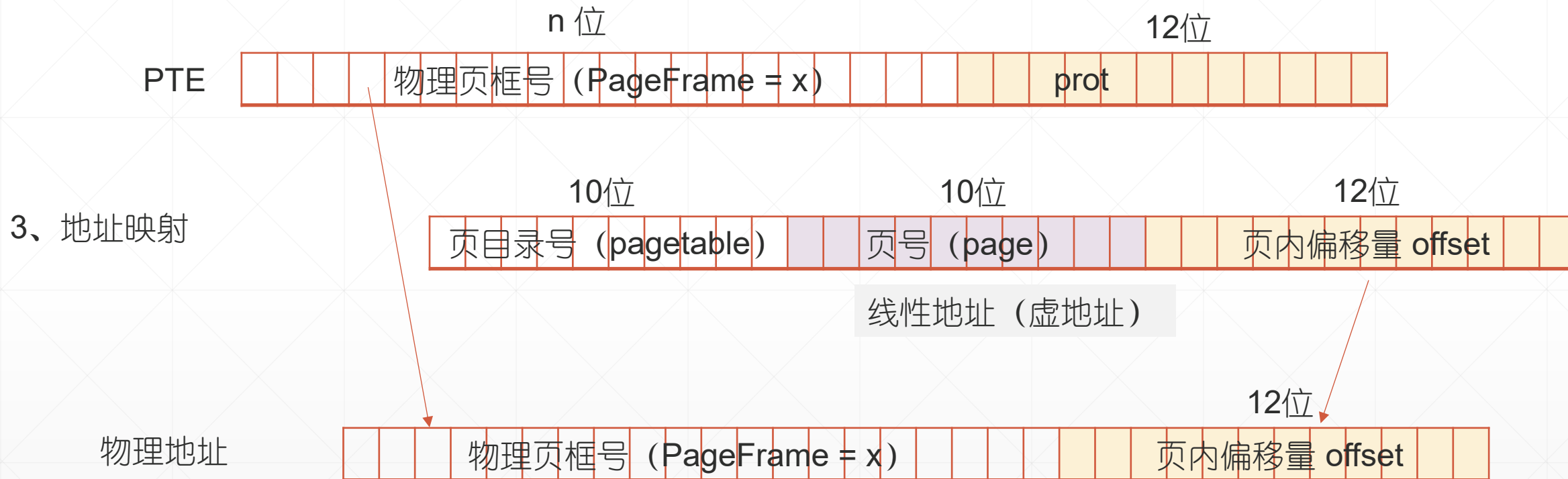
if(写内存单元 & R/W为0)

非法内存访问。访问的单元不可写，终止现运行进程。

if(CPU取指令 & 目标段X位是0, 不可执行)

非法内存访问。修改代码段、只读数据段，终止现运行进程。

页级地址映射和内存保护 (3)



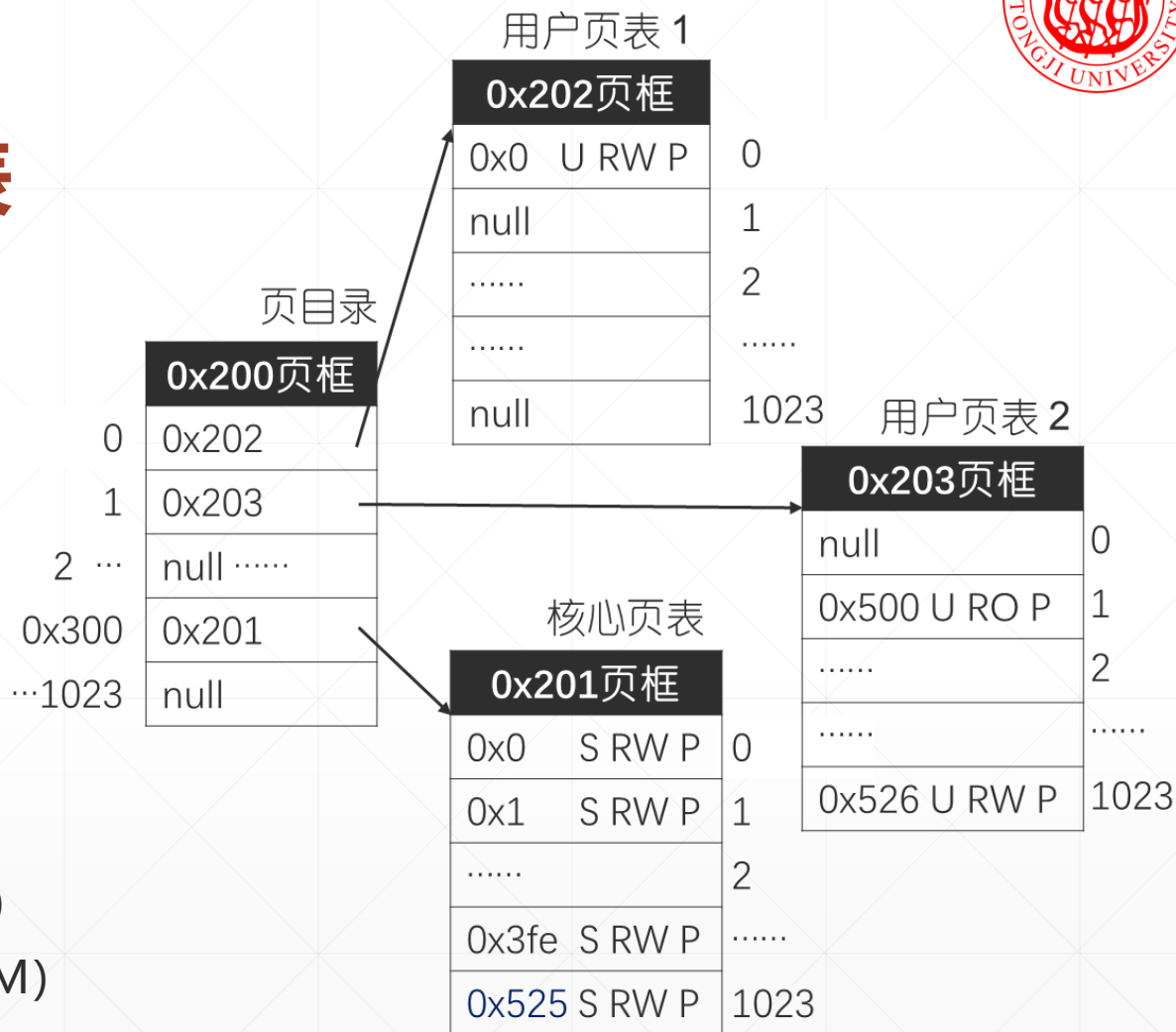
Unix V6++使用的二级页表

系统页表，包括

- 1张页目录，存放在0x200#物理页框
- 1张核心页表，存放在0x201#物理页框
- 2张用户页表，存放在0x202, 0x203#物理页框
- CR3是页表寄存器，值 0x200000，页目录的物理地址

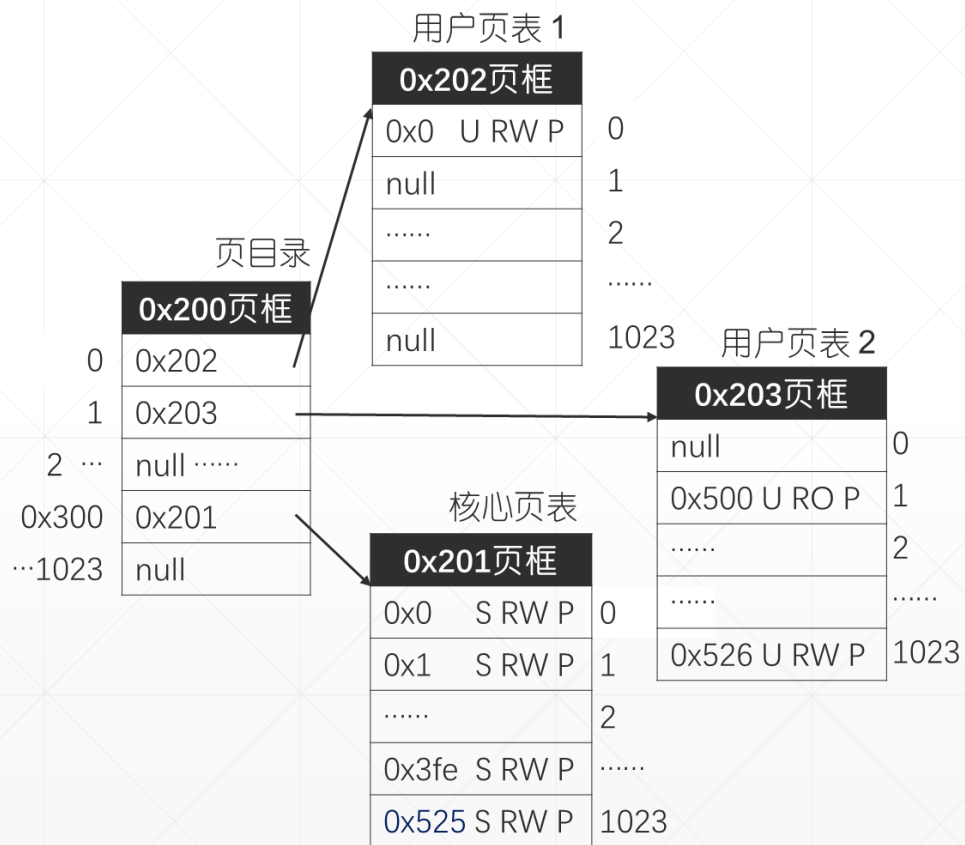
系统页表，登记现运行进程地址映射关系

- 0#页目录项，管理0#页表（第1张用户页表），映射[0,4M)
- 1#页目录项，管理1#页表（第2张用户页表），映射[4M,8M)
- 0x300#页目录项，管理768#页表（核心页表），映射[3G, 3G+4M)



例：现运行进程，1页代码，1页堆栈
x_caddr = 0x500000, p_addr = 0x525000

地址映射和内存保护



例：现运行进程，1页代码，1页堆栈
`x_caddr = 0x500000`, `p_addr = 0x525000`

- 代码段第1条指令。
 - `VA = 0x401000`, `PA = 0x500000`
- user结构
 - `VA = 0xC03ff000`, `PA = 0x525000`

写出MMU映射过程：

- 1、用户态 取值 `VA=0x401000`
 - `call main1`
 - 2、核心态 写 `VA= 0xC03ff000`
 - `mov %ebp, 0xC03ff000`
- 用户态
- 3、用户态 写 `VA=0x401000`
 - 4、用户态 读 `VA=0xC01a8000`

Unix V6++ 的相对虚实地址映射表

登记用户空间，每个逻辑页的属性和它在物理内存块中的相对偏移量。

相对表				
	/	/	/	/
				
	/	/	/	/
0x401	0	U	RO	P
0x402	1	U	RO	P
0x403	2	U	RO	P
0x404	1	U	RW	P
0x405	2	U	RW	P
0x406	3	U	RW	P
	全0			
0x7ff	4	U	RW	P
	base		prot	

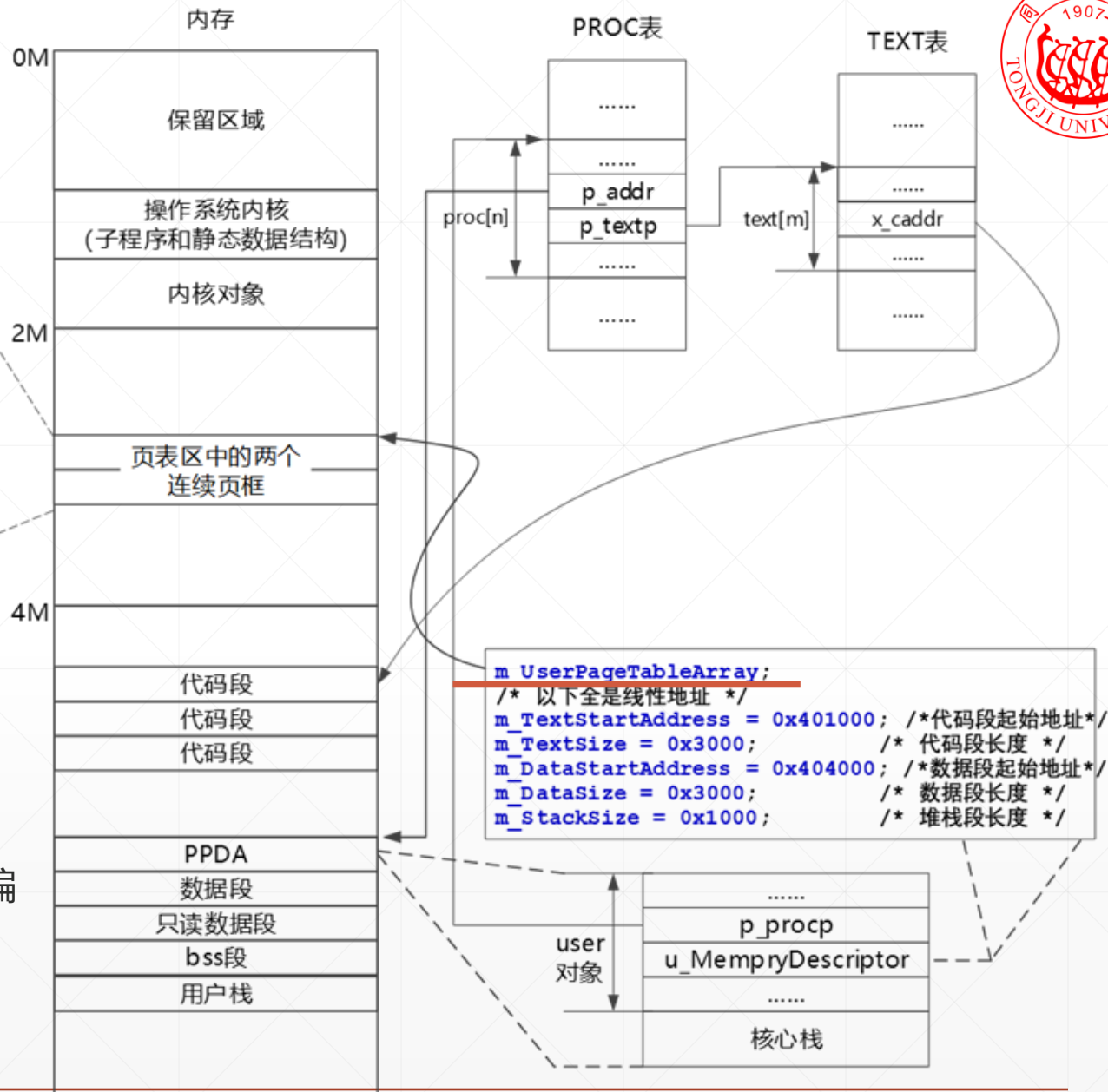
每个逻辑页，对应1个表项

prot是逻辑页的属性,12位

- RO, 代码; RW, 数据/堆栈
- P, 1 存在, 0 不存在
- U, 应用程序的页面

base是相对于起始页框的偏移量, 20位 (单位, 页框)

- RO, 相对于共享正文段
- RW, 可交换部分



选中目标进程后，为CPU建立新选中进程的地址映射关系

1、 $p_addr \gg 12$ ，填入**核心页表1023#PTE**的 base 字段，建立PPDA区的地址映射关系。

```
#define SwitchUStruct(p)\
    Machine::Instance().GetKernelPageTable().m_Entrys[Kernel::USER_PAGE_INDEX].m_PageBaseAddress \
        = (p)->p_addr / PageManager::PAGE_SIZE;\
    FlushPageDirectory();
```

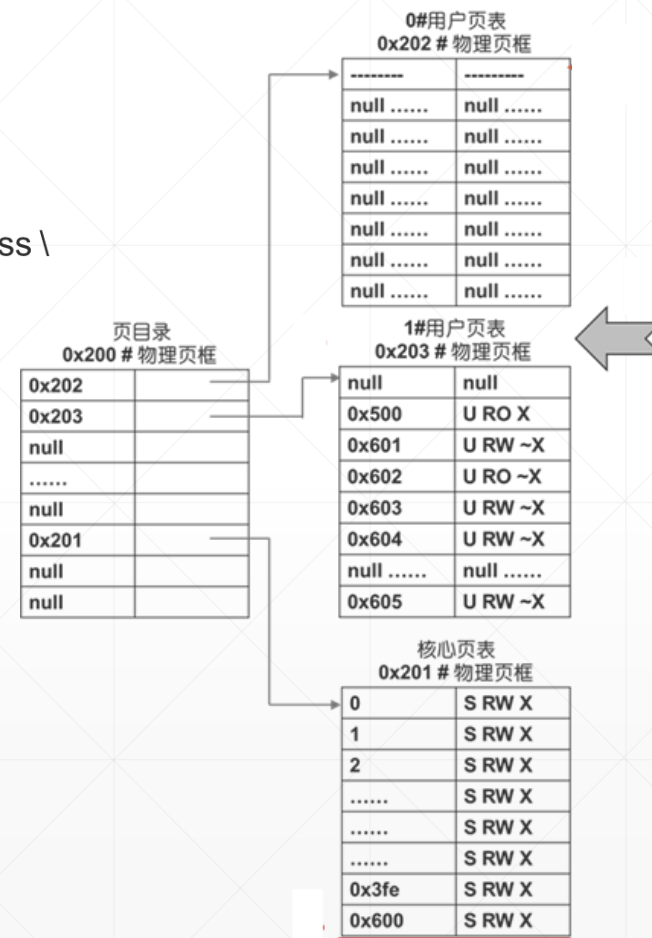
1023

PTE中的base

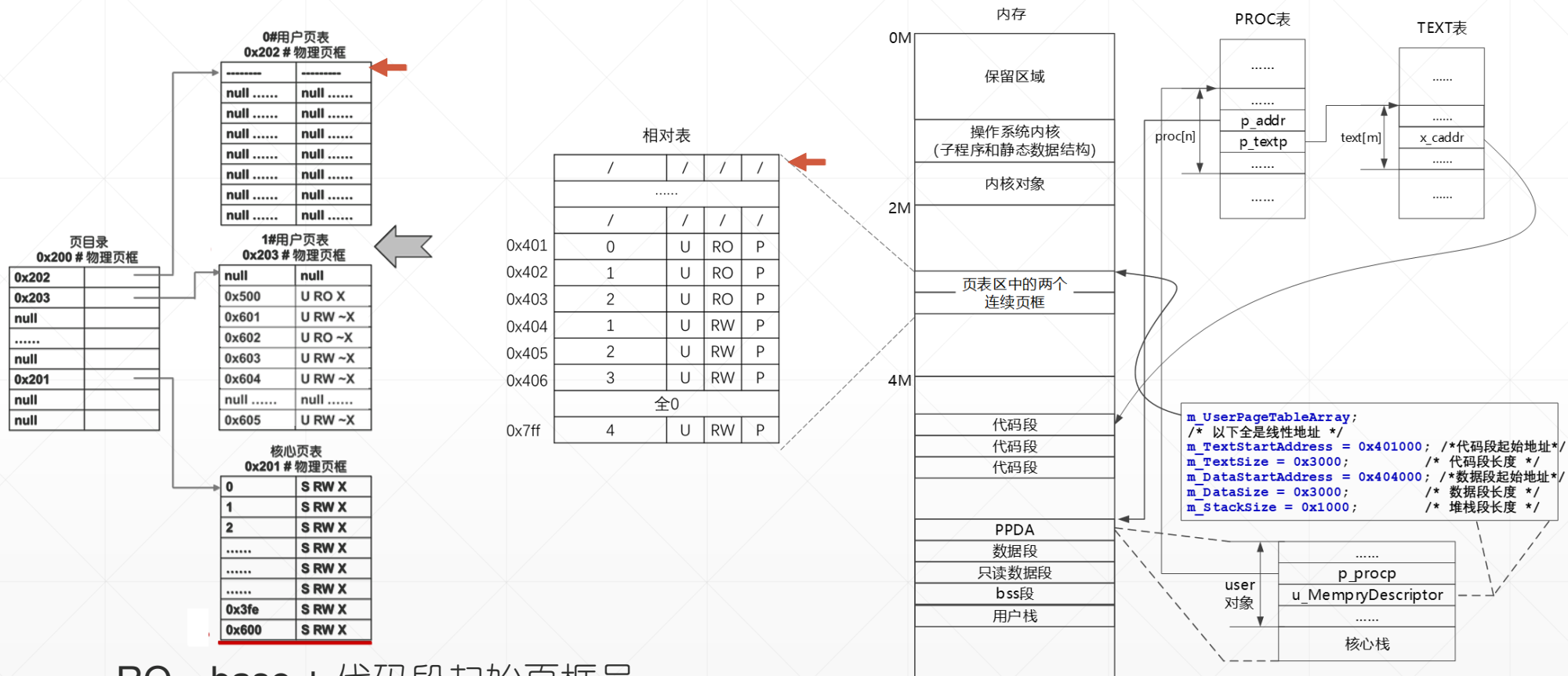
2、 User& u = Kernel::Instance().GetUser();

随后，访问user结构，可以得到新选中进程的相对表

3、刷新0x202、0x203#物理页框，建立新进程用户空间的地址映射关系



MapToPageTable()写系统页表



RO: base + 代码段起始页框号
RW: base + 可交换部分起始页框号